

مسار هندسة الرؤية الحاسوبية التطبيقية

الجهة	المشرف على المسار (الدور: مدير تنفيذي / مشرف / مستشار)
مقدم من	المتدرب — omori99100-dot
الفترة المشمولة	الأسبوع 7 — المرحلة 2: تحسين النموذج (Overfitting, Dropout, BatchNorm) — إتمام المرحلة
الحالة العامة	مكتمل ✅ — المرحلة 2 (PyTorch/CNN) منتهية بالكامل، جاهز للمرحلة 3
مستودع GitHub	github.com/omori99100-dot/cv-engineer-path

١. الملخص التنفيذي

أنجزت خلال هذا الأسبوع أهداف الأسبوع السابع والأخير من المرحلة الثانية: تشخيص ومعالجة Overfitting عبر مقارنة Train Loss و Test Loss عبر الـ epochs، ثم تطبيق أداتين مباشرتين للمعالجة — Dropout و Batch Normalization. بهذا تُختتم المرحلة الثانية كاملة (الأسابيع 4-7: من الشبكات العصبية اليدوية بـ NumPy، مرورًا بـ PyTorch و CNN، وصولًا لتقنيات تحسين التعميم)، والمتدرب جاهز للانتقال للمرحلة الثالثة (تطبيقات CV حقيقية: Classification، Detection، Segmentation).

الأداء المفاهيمي جيد بعد تصحيح أولي مهم (عكس اتجاه العلاقة بين Train Loss و Overfitting)، والنتائج العملية دعمت المبادئ النظرية بأرقام واضحة (تحسّن Test Loss مع Dropout رغم ارتفاع Train Loss). الأسبوع تميّز بأطول سلسلة أعطال تنفيذ تقنية متتالية منذ بداية المسار (كود ناقص، أخطاء إزاحة، أوامر PowerShell)، مفصلة بقسم الملاحظات.

٢. الأعمال المنجزة

2.1 تشخيص Overfitting (اليوم 1)

- فهم الفرق الجوهرى بين Training Loss (يقيس الحفظ) و Test/Validation Loss (يقيس التعميم الفعلي).
- تصحيح فهم أولي معكوس: الافتراض بأن Train Loss يرتفع عند حدوث Overfitting — صُحِّح بتوضيح أن Train Loss ينخفض بثبات دائمًا تقريبًا، وأن العلامة الفعلية هي توقف/ارتفاع Test Loss رغم استمرار انخفاض Train Loss.
- تدريب شبكة CNN من الأسبوع الماضي لـ 10 epochs (بدل 3)، ورصد فعلي لبداية Overfitting خفيف: تذبذب Test Loss بعد Epoch 4 تقريبًا رغم استمرار انخفاض Train Loss بثبات.

2.2 Dropout (اليوم 2 — جزء أول)

- فهم Dropout كآلية تُطفئ عشوائيًا نسبة من الخلايا أثناء التدريب فقط، لمنع اعتماد الشبكة المفرط على مسارات محددة.
- فهم دقيق (بمثال ملموس) لسبب توقف Dropout تلقائيًا وقت model.eval(): ضمان تنبؤات حتمية وقابلة لإعادة الإنتاج على نفس المدخل، بدل نتائج عشوائية متغيرة.
- تطبيق عملي وموازنة النتائج: Train Loss ارتفع (0.0828 مقابل 0.0309 بدون Dropout) بينما Test Loss تحسّن واستقر (0.0370 مقابل 0.0445)، مع نمط انخفاض أكثر ثباتًا — دليل عملي مباشر على مبدأ 'التضحية بالحفظ مقابل التعميم'.

2.3 Batch Normalization (اليوم 2 — جزء ثان)

- ربط مباشر بمبدأ Histogram Equalization/CLAHE من الأسبوع الثالث: تطبيع القيم لمدى موحد ومتوقع، لكن مطبقًا هنا على القيم الداخلية بين طبقات الشبكة أثناء التدريب بدل قيم بكسل الصورة.
- تطبيق nn.BatchNorm2d بعد كل طبقة Convolution وقبل دالة التنغيع (Conv → BatchNorm → ReLU → Pool)، مع تحسّن طفيف وملاموس بـ Test Loss (0.0331 مقابل 0.0370 مع Dropout فقط).

- فهم صحيح ومبرر (يربط تناظري مع تفاوت الإضاءة الحاد) لسبب كون أثر BatchNorm خفيفًا على شبكات صغيرة وحاسمًا على شبكات عميقة جدًا، حيث تتراكم مشكلة تفاوت مدى القيم عبر عشرات الطبقات المتتالية.

٣. مؤشرات الإنجاز

المهارة / المفهوم	التنفيذ	الفهم المفاهيمي
تشخيص Overfitting عبر Train مقابل Test Loss	✓ مؤكد بالكود	⚠ تحقق بعد عكس اتجاه أولي ثم صُحح
Dropout ومنطق التضحية بالحفظ	✓ مؤكد — أرقام موازنة واضحة	✓ متين — تفسير دقيق بمثال ملموس
سبب توقف Dropout وقت eval()	✓ لا ينطبق (سؤال تحليلي)	✓ متين بعد توضيح إضافي
BatchNorm وربطها بـ CLAHE السابق	✓ مؤكد بالكود	✓ متين — ربط تناظري صحيح
تنفيذ الكود (بنية الملف والإزاحة)	⚠ سلسلة أعطال طويلة قبل النجاح	لا ينطبق (مهارة تقنية لا مفهوم)
المرحلة 2 ككل (تقييم شامل)	✓ مؤكد — أربعة أسابيع مكتملة	✓ متين ومتصاعد عبر الأسابيع

٤. الصعوبات والأعطال التي واجهت التنفيذ

هذا الأسبوع تضمن أطول سلسلة أعطال تنفيذ تقنية متتالية منذ بداية المسار، إلى جانب ملاحظة مفاهيمية واحدة:

- عكس أولي لاتجاه العلاقة بين Overfitting و Train Loss (افتراض أن Train Loss يرتفع بدل الفهم الصحيح لاستمرار انخفاضه) — صُحح بمثال منطقي مباشر قبل التنفيذ.
 - KeyboardInterrupt أثناء استيراد المكتبات — مقاطعة غير مقصودة أثناء انتظار طبيعي، لا علاقة له بالكود.
 - خطأ PowerShell (Unexpected Token) ناتج عن تنسيق غير صحيح لأمر التشغيل.
 - خطأ 'file not found' بسبب تشغيل الأمر من مجلد أب (cv_engineer_path) بدل مجلد week7 المطلوب.
 - ملف يحتوي تعريف الكلاس فقط دون كود تشغيل فعلي (نسخ جزئي للكود المُعطى، تكرار لنمط عُلق عليه سابقًا بالأسبوع الثاني).
 - NameError ناتج عن لصق سطر استدعاء الكلاس داخل جسم الكلاس نفسه بدل خارجه (خطأ إزاحة/indentation).
 - IndentationError ثانٍ بنفس السبب — عولج نهائيًا بتوفير الملف كاملاً جاهزاً للنسخ المباشر بدل التعديل اليدوي الجزئي.

٥. التقييم (رأي صريح)

- الجاهزية التقنية للانتقال إلى المرحلة 3 (تطبيقات CV حقيقية) — جاهز — المرحلة 2 كاملة مبنية على فهم تراكمي موثّق، لا حفظ منعزل.
 - جودة تنفيذ الكود (بنية الملف، الإزاحة، أوامر التشغيل) — أضعف نقطة هذا الأسبوع — سلسلة أعطال طويلة أطالت وقت الجلسة دون علاقة بالفهم المفاهيمي.
 - الفهم المفاهيمي (Overfitting, Dropout, BatchNorm) — جيد بعد تصحيح واحد أساسي، مدعوم بأرقام تجريبية واضحة ومقارنات صحيحة.
- التوصية: قبل بدء المرحلة 3، خذ 10 دقائق لمراجعة عادتين برمجتين تكررنا عبر عدة أسابيع: (1) نسخ كود مُعطى بالكامل وليس جزءاً منه، و (2) التحقق من مستوى الإزاحة (indentation) قبل التشغيل، خصوصاً عند إضافة كود إلى ملف يحتوي على تعريف class. هاتان العادتان — لا صعوبة المفاهيم — كانتا مصدر معظم وقت التشخيص المُهدر هذا الأسبوع والأسبوع الثاني.
- تقييم شامل للمرحلة 2 (الأسابيع 4-7): بدأت بشبكة NumPy يدوية كاملة (الأسبوع 4)، مروّزاً بـ PyTorch و CNN (الأسبوعين 5-6) بدقة وصلت 98.29% على MNIST، وانتهت بتقنيات تحسين التعميم (الأسبوع 7). النمط عبر الأربعة أسابيع تصاعدي بوضوح: تصحيحات

مفاهيمية أقل تكراراً، وتفسيرات أكثر استقلالية مع كل أسبوع. المرحلة التالية (تطبيقات CV حقيقية) هي الاختبار الفعلي لتطبيق هذا الأساس على مشاكل هندسية واقعية.

٦. تذكير بالخطة العامة

لمنع التشتت — موقع هذا الأسبوع ضمن المسار الكامل (16 أسبوعاً):

- الأسبوع 1-3: الأساسيات الصورية (OpenCV) ← مكتملة بالكامل ☒.
- الأسبوع 4-7: أساسيات التعلم العميق (PyTorch, CNN) ← مكتملة بالكامل ☒.
- الأسبوع 8-12: تطبيقات CV حقيقية (Classification, Detection/YOLO, Segmentation) ← المرحلة التالية مباشرة.
- الأسبوع 13-16: مشروع 'End-to-End'، Kaggle، تجهيز المحفظة والتوظيف.